```
"pratikms"), 0755)
    if err != nil && !os.IsExist(err) {
        panic(err)
    }
    must(ioutil.WriteFile(filepath.
Join(pids, "pratikms/pids.max"), []
byte("20"), 0700))
    // Remove the new cgroup after
container exits
    must(ioutil.WriteFile(filepath.
Join(pids, "pratikms/notify_on_
release"), []byte("1"), 0700))
    must(ioutil.WriteFile(filepath.
Join(pids, "pratikms/cgroup.procs"), []
byte(strconv.Itoa(os.Getpid())), 0700))
}

func child() {
    fmt.Printf("Running %v as PID
%d\n", os.Args[2:], os.Getpid())

    cg()

    cmd := exec.Command(os.Args[2],
os.Args[3:]...)
    cmd.Stdin = os.Stdin
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr

    must(syscall.Sethostname([]
byte("container")))
    must(syscall.Chroot("/home/lubuntu/
Projects/make-sense-of-containers/
lubuntu-fs"))
    must(syscall.Chdir("/"))
    must(syscall.Mount("proc", "proc",
"proc", 0, ""))

    must(cmd.Run())
    must(syscall.Unmount("/proc", 0))
}

func run() {
    cmd := exec.Command("/proc/self/
exe", append([]string{"child"},
os.Args[2:]...)...)
    cmd.Stdin = os.Stdin
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr

    cmd.SysProcAttr = &syscall.
SysProcAttr{
        Cloneflags:   syscall.
CLONE_NEWUTS | syscall.CLONE_NEWPID |
syscall.CLONE_NEWNS,
        Unshareflags: syscall.CLONE_
NEWNS,
    }

    must(cmd.Run())
}
func main() {
    switch os.Args[1] {
    case "run":
        run()
    case "child":
        child()
    default:
        panic("I'm sorry, what?")
    }
}
```

Again, as stated before, this is in no way a production-ready code. I do have some hard-coded values in it, like the value of the path to the file system, and also the hostname of the container.

If you wish to play around with the code, you can get it from my GitHub repo *(https://github.com/pratikms/demystifying-containers)*. But, at the same time, I do believe this is a wonderful exercise to understand what goes on behind the scenes when we run that *docker run <image>* command in our terminal. It introduces us to some of the important OS concepts that containers generally leverage, like namespaces, layered file systems, Cgroups, etc.

Containers are important – and their prevalence in the job market is incredible. With cloud, Docker and Kubernetes getting closer with each new day, the demand for containers will only grow.

Going forward, it will be imperative to understand the inner workings of a container. With this knowledge, we can attempt to look at the magic that goes on in a container orchestrator like Kubernetes. But that's for a different article! END

By: Pratik Shivaraikar

The author is an open source software (OSS) enthusiast, evangelist and contributor with varied experience of working in the storage, security and wireless domains. He actively works on Go, Python, Node.js, PHP, and more. His hobbies include playing chess, bowling, and motorcycling. He writes at *https://blog.pratikms.com*.